TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



ĐỀ TÀI CUỐI KỲ

SINH MẪU KIỂM TRA TỰ ĐỘNG CHO MẠCH TỔ HỢP VÀ TUẦN TỰ

Automatic Test Pattern Generation Algorithms
for Combinational and Sequential Circuits

Môn học: Kỹ thuật DFT và kiểm thử

Nhóm SVTH: Nhóm 4

Phan Ngọc Tuấn Nguyên
MSSV: 23119178

Hà Quang Huy
MSSV: 23119147

Võ Trung Nguyên
MSSV: 23119180

Phạm Trọng An Nam
MSSV: 23119174

Nguyễn Thị Thúy Hằng
MSSV: 23119142

Nguyễn Thị Thanh Tuyền
MSSV: 23119222

GVHD: **Nguyễn Văn Thành Lộc**

TP. Hồ Chí Minh, 10/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH



ĐỀ TÀI CUỐI KỲ

SINH MẪU KIỂM TRA TỰ ĐỘNG CHO MẠCH TỔ HỢP VÀ TUẦN TỰ

Automatic Test Pattern Generation Algorithms
for Combinational and Sequential Circuits

Môn học: Kỹ thuật DFT và kiểm thử

Nhóm SVTH: Nhóm 4
Phan Ngọc Tuấn Nguyên
MSSV: 23119178
Hà Quang Huy
MSSV: 23119147
Võ Trung Nguyên
MSSV: 23119180
Phạm Trọng An Nam
MSSV: 23119174
Nguyễn Thị Thúy Hằng
MSSV: 23119142
Nguyễn Thị Thanh Tuyền
MSSV: 23119222

GVHD: **Nguyễn Văn Thành Lộc**

TP. Hồ Chí Minh, 10/2026

Mục lục

Danh mục viết tắt **ii**

1 Giới thiệu **1**

2 Nguyên lý ATPG **1**

3 D-algorithm **2**

4 Thuật toán PODEM **4**

5 ATPG cho mạch tuần tự **5**

6 Cài đặt PODEM **6**

7 Kết quả và kiểm chứng **7**

7.1 Môi trường, mạch thử và danh sách lỗi 7

7.2 Phương pháp kiểm chứng 7

7.3 Kết quả trên c17 8

7.4 Mạch tuần tự 8

7.5 Nhận xét và giới hạn 9

8 Kết luận **9**

Tài liệu tham khảo **10**

Danh mục viết tắt

Viết tắt	Tiếng Anh	Tiếng Việt
DFT	Design for Testability	Thiết kế hỗ trợ kiểm thử
ATPG	Automatic Test Pattern Generation	Sinh mẫu kiểm tra tự động
PI	Primary Input	Ngõ vào chính
PO	Primary Output	Ngõ ra chính
SA0	Stuck-at-0	Lỗi kẹt mức 0
SA1	Stuck-at-1	Lỗi kẹt mức 1
PODEM	Path-Oriented Decision Making	Thuật toán sinh mẫu kiểm tra tự động định hướng theo đường đi
SCOAP	Sandia Controllability/Observability Analysis Program	Độ đo khả năng điều khiển và quan sát
DFF	D Flip-Flop	Phần tử nhớ D

1. Giới thiệu

Kiểm chứng thiết kế (verification) đánh giá thiết kế có đáp ứng đặc tả hay không, trong khi kiểm thử sau sản xuất (manufacturing testing) nhằm phát hiện các chip có khuyết tật chế tạo. Một thiết kế đúng vẫn có thể tạo ra sản phẩm lỗi, do đó hai hoạt động này bổ sung cho nhau. Thay vì kiểm tra mọi tổ hợp đầu vào, kiểm thử cấu trúc sử dụng mô hình lỗi để xây dựng các mẫu kiểm tra có mục tiêu. Sinh mẫu kiểm tra tự động (Automatic Test Pattern Generation – ATPG) tìm các kích thích làm cho đáp ứng của mạch lỗi khác mạch tốt tại ngõ ra quan sát được [1].

Đề tài tập trung giải thích nguyên lý ATPG, cơ chế của D-algorithm và PODEM, đồng thời trình bày những khó khăn khi áp dụng ATPG cho mạch tuần tự. Phần thực hành đã cài đặt PODEM bằng Python, minh họa trên mạch c17 và đối chiếu mẫu sinh ra bằng mô phỏng lỗi. Lỗi `11/SA0` được chọn làm ví dụ chung để so sánh quá trình tìm kiếm bằng tay với chương trình.

Phạm vi chính là mô hình lỗi kẹt mức logic đơn (single stuck-at fault). Đối với mạch tuần tự nhỏ, nhóm khảo sát thiết kế quét đầy đủ (full scan) và trải khung thời gian (time-frame expansion). Các phần tiếp theo lần lượt trình bày cơ sở lý thuyết, thuật toán, cách cài đặt, kết quả kiểm chứng và kết luận; những giới hạn thực nghiệm sẽ được nêu theo kết quả thực tế.

2. Nguyên lý ATPG

Mô hình lỗi và điều kiện phát hiện. Lỗi kẹt mức (stuck-at) buộc một đường tín hiệu luôn bằng 0 (SA0) hoặc 1 (SA1); giả thiết lỗi đơn xét từng lỗi riêng. Lỗi trên thân dây (stem) tác động mọi nhánh đi ra, còn lỗi nhánh (branch) chỉ tác động một ngõ vào cổng. Mô hình này đơn giản, phù hợp ATPG mức cổng nhưng không mô tả mọi khuyết tật: lỗi cầu nối (bridging) liên quan các dây nối ngoài ý muốn, còn lỗi trễ (delay) liên quan đáp ứng không kịp thời gian [1, Ch. 1]. Để phát hiện lỗi phải *kích hoạt* giá trị đối nghịch tại vị trí lỗi, *lan truyền* khác biệt đến PO và *justify* các yêu cầu bằng gán PI nhất quán. Logic năm giá trị dùng $0, 1, X, D, D'$, với $D = (1, 0)$, $D' = (0, 1)$ theo thứ tự (mạch tốt, mạch lỗi); X là chưa biết. Trên c17, theo thứ tự PI $(1, 2, 3, 6, 7)$, mẫu $(X, 1, 0, X, X)$ cho lỗi `11/SA0` tạo $11 = D$, $16 = D'$, $10 = 1$, $22 = D$. Mọi cách điền ba giá trị X đều phát hiện lỗi này.

Danh sách và rút gọn lỗi. Nếu chỉ xét hai lỗi trên mỗi net, c17 có $2(5+6) = 22$ lỗi; thêm lỗi nhánh sẽ thay đổi mẫu số. Gọi $T(f)$ là tập vector phát hiện lỗi f. Hai lỗi tương đương về phát hiện khi $T(f_1) = T(f_2)$; tương đương đáp ứng mạnh hơn, yêu cầu cùng đáp ứng lỗi ở mọi PI. Với NAND $z = \overline{ab}$, a/SA0, b/SA0 và z/SA1 tương đương tại cổng. Trên c17, net 1 chỉ nối đến cổng 10 nên có thể giữ một đại diện cho 1/SA0 và 10/SA1: cả hai buộc $10 = 1$, do đó có cùng đáp ứng PO trên cả 32 vector. Không áp dụng quy tắc cục bộ này trực tiếp cho stem có nhiều nhánh. Với dominance, điều kiện $T(f_a) \subseteq T(f_b)$ cho phép giữ f_a: mọi mẫu phát hiện f_a cũng phát hiện f_b. Ví dụ NAND riêng có $T(a/\text{SA1}) = \{01\}$ nằm trong $T(z/\text{SA0}) = \{00, 01, 10\}$. Cần ghi chiều bao hàm để tránh nhầm lỗi đại diện [1, pp. 12–14].

Mô phỏng và độ bao phủ. Mô phỏng lỗi (fault simulation) kiểm tra một bộ mẫu trên mạch tốt và mạch lỗi; kiểu serial xét từng lỗi, còn kiểu parallel có thể mã hóa nhiều mạch lỗi vào các bit của một từ máy. Fault dropping loại lỗi đã được phát hiện khỏi danh sách còn phải xét. Với danh sách lỗi gốc F, độ bao phủ là $FC = |F_{\text{det}}|/|F| \times 100\%$; phải đếm mỗi lỗi một lần và ghi rõ stem/branch, trước/sau collapsing. Nếu báo cáo test coverage loại các lỗi đã chứng minh không thể phát hiện, mẫu số là $|F| - |F_{\text{untestable}}|$. Không tìm được mẫu trong giới hạn tài nguyên là `ABORTED`, không phải `UNTESTABLE`. Trong mạch tổ hợp, lỗi không có vector phát hiện được gọi là untestable; lỗi dư thừa là một trường hợp liên quan logic dư thừa. Theo quy ước nhóm, điền $X = 0$ trước mô phỏng nhị phân và kiểm tra lại mẫu.

Khả năng điều khiển, quan sát và SCOAP. SCOAP ước lượng chi phí đặt net về 0/1 ($CC0, CC1$) và quan sát nó (CO); số nhỏ hơn biểu thị dễ hơn, không phải xác suất. Đặt $CC0 = CC1 = 1$ tại PI, $CO = 0$ tại PO. Với $z = \text{NAND}(a, b)$ [1, pp. 41–42]:

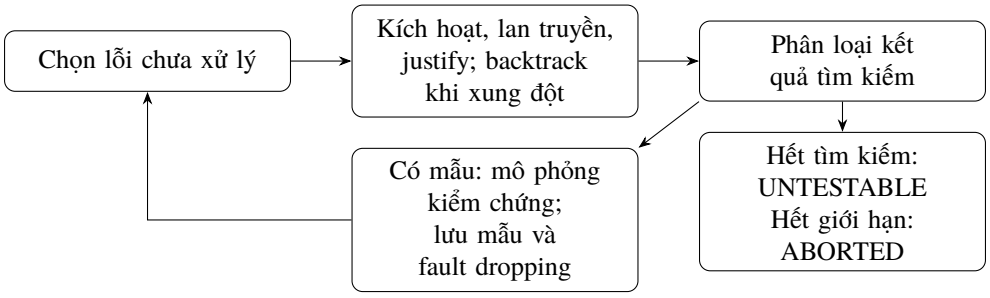
$$CC0(z) = CC1(a) + CC1(b) + 1, \quad CC1(z) = \min(CC0(a), CC0(b)) + 1,$$
$$CO(a \to z) = CO(z) + CC1(b) + 1, \qquad CO(s) = \min_{\text{nhánh } j} CO(s_j).$$

Tính CC từ PI đến PO rồi CO theo chiều ngược lại thu được Bảng 2.1. Chẳng hạn $CC0(11) = 1 + 1 + 1 = 3$, $CC1(11) = 2$; $CO(11) = \min(3 + 1 + 1, 3 + 1 + 1) = 5$. Do fanout hội tụ lại (reconvergence), SCOAP bỏ qua tương quan giữa các nhánh; đây là heuristic định hướng tìm kiếm, không chứng minh một lỗi có thể phát hiện.

Bảng 2.1: SCOAP c17, tính trên net/stem theo quy tắc trong giáo trình.

Net	1	2	3	6	7	10	11	16	19	22	23
$CC0$	1	1	1	1	1	3	3	4	4	5	5
$CC1$	1	1	1	1	1	2	2	2	2	4	5
CO	5	6	5	7	6	3	5	3	3	0	0

Phương pháp sinh mẫu. Sinh mẫu ngẫu nhiên ít tốn công tìm kiếm nhưng khó xử lý lỗi khó phát hiện. ATPG tất định hướng tìm kiếm vào từng lỗi: D-algorithm (Roth, 1966) thao tác trên các đường tín hiệu; PODEM (Goel, 1981) quyết định tại PI; FAN (Fujiwara và Shimono, 1983) khai thác cấu trúc fanout để giảm tìm kiếm [1, Ch. 4]. Nhóm chọn PODEM để trình bày objective, backtrace, imply và backtrack; chưa mặc định đây là thuật toán nhanh nhất. Hình 2.1 mô tả luồng xử lý chung.

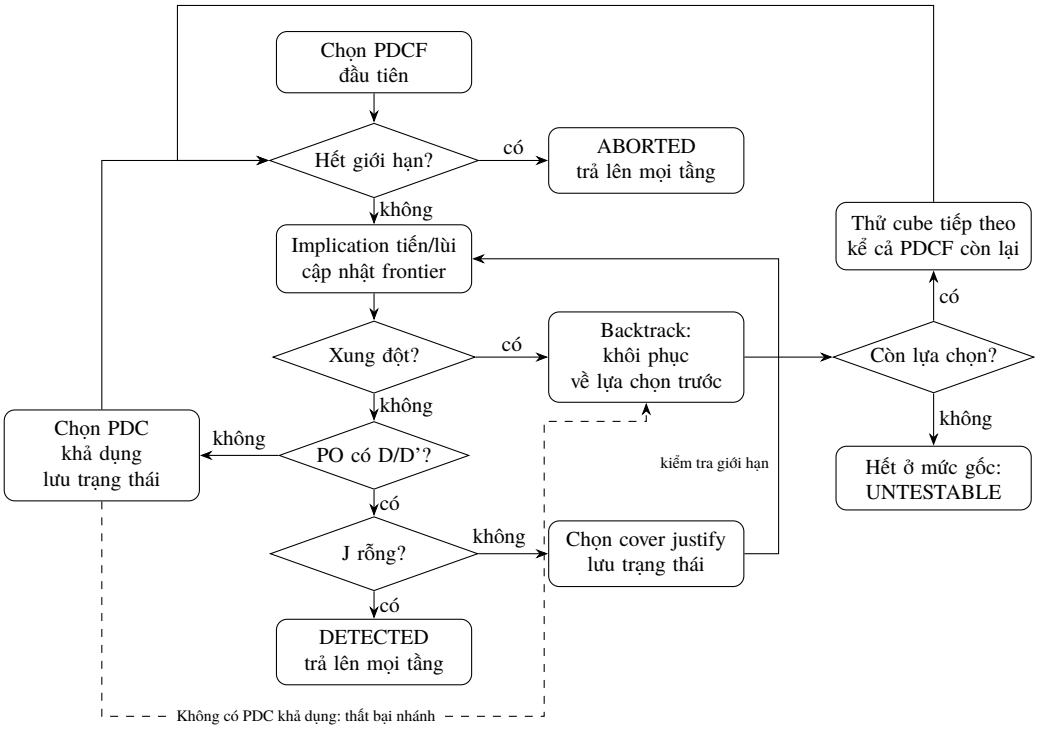


Hình 2.1: Luồng ATPG tất định và kiểm chứng mẫu.

3. D-algorithm

D-calculus và các cube. D-algorithm của Roth (1966) biểu diễn sai khác giữa mạch tốt và mạch lỗi bằng $D = (1, 0)$, $D' = (0, 1)$; $0 = (0, 0)$, $1 = (1, 1)$ và X chưa biết [2]. Đánh giá cổng riêng trên hai thành phần rồi ghép kết quả: $D \wedge 1 = D$, $D \wedge D' = 0$, $D \wedge X = X$, $\text{NAND}(D, 1) = D'$. Nhóm lập bảng đầy đủ cho tám loại cổng và dùng làm dữ liệu kiểm thử chung cho chương trình. Với NAND $z = \overline{ab}$, singular cover gồm $(0, X, 1)$, $(X, 0, 1)$, $(1, 1, 0)$. PDCF cho $z/\text{SA0}$ là $(0, X, D)$ hoặc $(X, 0, D)$; PDC như $(D, 1, D')$ đưa sai khác qua cổng. D-intersection kết hợp các yêu cầu cùng net: $X \cap v = v$, $v \cap v = v$; hai giá trị xác định khác nhau gây xung đột khi cực D/D' đã cố định.

Tìm kiếm và justification. D-frontier chứa cổng có đầu vào D/D' nhưng đầu ra X; J-frontier chứa cổng có đầu ra đã gán nhưng đầu vào chưa justify được. Thuật toán chọn PDCF, thực hiện implication tiến/lùi, chọn PDC để đưa lỗi ra PO, rồi dùng singular cover justify các yêu cầu nội bộ. Khi xung đột, khôi phục trạng thái và thử cube khác. Chỉ thành công khi sai khác tới PO *và* mọi yêu cầu đều nhất quán, được thực hiện bởi PI. Lưu đồ ở Hình 3.1.



Hình 3.1: Luồng D-algorithm; khôi phục cả gán net và frontier khi quay lui.

Lưu đồ bắt đầu bằng việc chọn PDCF để kích hoạt lỗi, sau đó implication cập nhật giá trị các net và frontier. Nếu sai khác chưa đến ngõ ra, thuật toán chọn PDC để lan truyền D/D' đến PO và dùng cover để justification các giá trị đầu vào còn thiếu. Khi gặp xung đột, thuật toán khôi phục trạng thái trước đó để thử lựa chọn khác. Kết quả là `DETECTED` khi sai khác đến PO và J-frontier rỗng; nếu dừng do chạm giới hạn tìm kiếm thì trả về `ABORTED`.

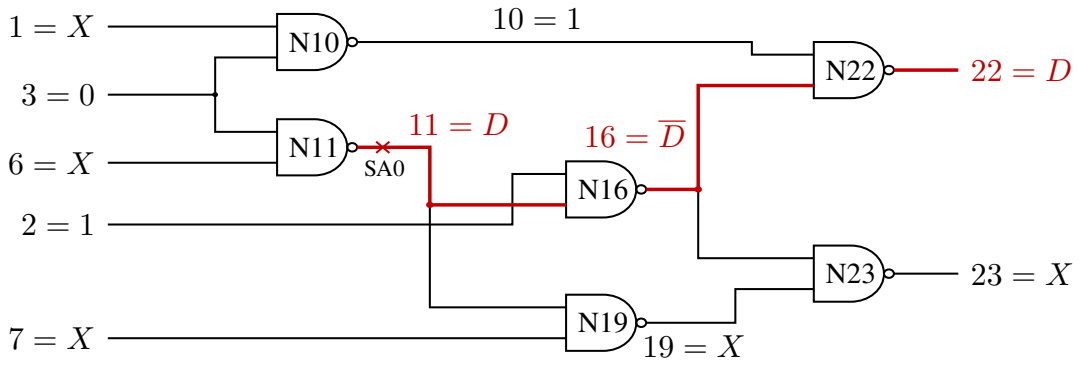
Ví dụ c17, lỗi stem $11/\mathrm{SA0}$**.** Bảng 3.1 ghi bốn lần chọn cube; không đếm các lần implication.

Bảng 3.1: Chạy tay D-algorithm: J ghi các net đầu ra chưa justify.

Bước	Cube trên các net	Implication chính	J
1	PDCF $(3, 6, 11) = (X, 0, D)$	$11 = D$	$\varnothing$
2	PDC $(2, 11, 16) = (1, D, D')$	$16 = D'$	$\varnothing$
3	PDC $(10, 16, 22) = (1, D', D)$	$22 = D$	$\{10\}$
4	Cover $(1, 3, 10) = (X, 0, 1)$	Gán PI $3 = 0$	$\varnothing$

Ở bước 3, $22 = D$ chưa đủ kết luận thành công vì net 10 còn cần justification. Theo thứ tự PI $(1, 2, 3, 6, 7)$, thu được $(X, 1, 0, 0, X)$, điền $X = 0$ thành 01000. Bốn cách điền đều cho

$22_{\text{tốt}} = 1$, $22_{\text{lỗi}} = 0$. Sau kiểm chứng có thể bỏ ràng buộc $6 = 0$, được $(X, 1, 0, X, X)$; đây là bước tổng quát hóa cube, không phải một quyết định tìm kiếm.



Hình 3.2: Mạch c17 với cube sau tổng quát hóa, $6 = X$; đường quan sát $11 = D \to 16 = D' \to 22 = D$.

Nhận xét. Quyết định trên net nội bộ tạo thêm công việc justification; fanout hội tụ có thể làm các yêu cầu cục bộ xung đột hoặc che sai khác. Trong c17, ảnh hưởng net 11 đi qua 16 và 19 rồi hội tụ tại 23. Tìm kiếm có thể tăng theo cấp số mũ trong trường hợp xấu.

4. Thuật toán PODEM

Nguyên lý. PODEM (Goel, 1981) chỉ ra quyết định nhị phân tại PI; các net nội bộ được suy ra bằng mô phỏng, nên không cần J-frontier như D-algorithm [3, 1]. Cây tìm kiếm trên n PI có tối đa 2^n vector đầy đủ; trường hợp xấu vẫn cần tìm kiếm theo hàm mũ. *Objective* chọn $(l, 1 - s)$ để kích hoạt lỗi $l/\text{SA}s$; sau kích hoạt, chọn đầu vào X của D-frontier về giá trị không điều khiển (1 cho AND/NAND, 0 cho OR/NOR). *Backtrace* đưa mục tiêu về PI chưa gán, đảo bit qua NAND/NOR/NOT; *imply* tính lại toàn mạch bằng logic $0, 1, X, D, D'$. Phiên bản của nhóm chọn cổng theo topo ổn định và đầu vào X đầu tiên trong netlist, không dùng SCOAP.

Xử lý cổng XOR/XNOR trong PODEM. Đặt $y = x_1 \oplus \cdots \oplus x_n \oplus q$, với $q = 0$ cho XOR, $q = 1$ cho XNOR. Backtrace mục tiêu $y = v$: nếu chỉ còn một đầu vào X, giá trị cần tại đó là $v \oplus q \oplus p$, trong đó p là parity các bit mạch tốt đã biết ($D \mapsto 1$, $D' \mapsto 0$). Nếu còn nhiều X, chọn X đầu tiên và thử 0, imply rồi tính lại. Objective lan truyền cũng ưu tiên X đầu tiên bằng 0; XOR/XNOR không có giá trị điều khiển. Ví dụ $D \oplus 0 = D$, $D \oplus 1 = D'$, nhưng $D \oplus D = 0$: nhiều nhánh lỗi có thể triệt tiêu. XNOR đảo kết quả XOR đúng một lần. Quy tắc này đã được cài đặt và kiểm thử trong chương trình (Chương 6).

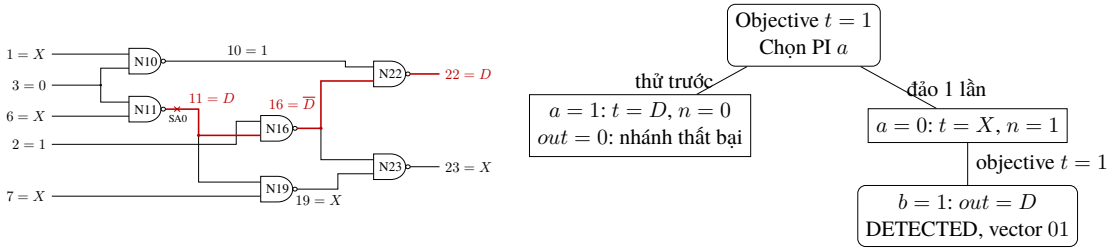
Dừng và quay lui. Có D/D' ở PO thì thành công. Khi lỗi đã kích hoạt nhưng không còn đường X từ D-frontier tới PO, nhánh thất bại; có X-path chỉ là điều kiện cần. Frontier rỗng trước kích hoạt chưa đủ để loại nhánh. Mã giả: *imply* $\to$ *kiểm tra PO/khả thi* $\to$ *objective/backtrace* $\to$ *thử PI* $b \to$ *nếu thất bại, kiểm tra giới hạn rồi thử* $1 - b \to$ *nếu cả hai thất bại, trả PI về X và quay về cha*. Xóa quyết định con trước khi đảo cha. Đếm backtrack bằng số phép đảo PI; hết cây mới kết luận UNTESTABLE, hết giới hạn là ABORTED.

c17, lỗi stem $11/\text{SA0}$**.** Theo thứ tự PI $(1, 2, 3, 6, 7)$, quá trình tìm kiếm cho Bảng 4.1.

Vì $3 = 0$ nên NAND11 tốt bằng 1; $2 = 1$ truyền D qua NAND16 thành D', rồi qua NAND22 thành D. Pattern $X10XX$, 0 backtrack; 8/8 cách điền X phát hiện lỗi tại 22. Điền X=0 được 01000, PO tốt $(1, 1)$, lỗi $(0, 0)$. PO23 còn X trong trace là xấp xỉ bảo thủ.

Bảng 4.1: Hai phép gán PI; các net chưa liệt kê giữ X.

Bước	Objective; PI	Net xác định sau imply	D-frontier
1	$(11, 1); 3 = 0$	$10 = 1, 11 = D$	$\{16, 19\}$
2	$(2, 1); 2 = 1$	$10 = 1, 11 = D, 16 = D', 22 = D$	$\{19, 23\}$



Hình 4.1: Trái: c17 ở trạng thái cuối. Phải: cây quyết định mạch phụ, thử $a = 1$ rồi đảo $a = 0$.

Ví dụ quay lui. Mạch phụ $t = a + b$, $n = \overline{a}$, $out = tn$, lỗi stem t/SA0: thử $a = 1$ cho $(t, n, out) = (D, 0, 0)$, nhánh thất bại với mọi b; đảo $a = 0$ cho $(X, 1, X)$; chọn lại objective $(t, 1)$, gán $b = 1$ cho $(D, 1, D)$. Kết quả 01, ba hàng gán/đảo, một backtrack. Giới hạn 0 phải ABORTED trước phép đảo, không phải UNTESTABLE. Hai ví dụ trên được lưu thành golden trace bảy cột để đối chiếu chương trình.

So sánh và đối chiếu. Trace tay D-algorithm (Chương 3) có 4 lần chọn cube (không đếm implication), 3 PI được gán, 0 backtrack, pattern $X100X$; sau tổng quát hóa được $X10XX$. PODEM có 2 phép gán PI, 0 backtrack. Hai đơn vị đếm khác nhau, không suy ra tỷ lệ tăng tốc. D-algorithm cần justification các quyết định nội bộ; PODEM dùng backtrace và imply. FAN cải tiến lựa chọn bằng cấu trúc fanout và multiple backtrace [4]; nhóm chưa cài FAN. Bộ kiểm chứng lý thuyết duyệt 32 vector c17, 4 vector mạch phụ và 22 lỗi stem; đạt các kỳ vọng trên. Trace bảy cột của chương trình có giá trị net khớp golden trace; chương trình ghi thao tác vừa làm, golden trace ghi hành động kế tiếp, nên bộ xuất trace chuẩn hóa nhãn theo golden. Bộ mô phỏng lỗi xác nhận cube c17 8/8 và mẫu phụ 01; giới hạn 0/1 lần lượt cho ABORTED/DETECTED. Số liệu chỉ áp dụng heuristic đã công bố.

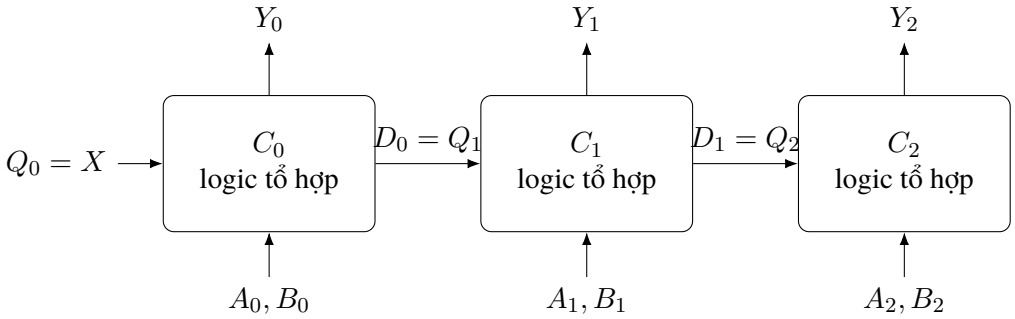
5. ATPG cho mạch tuần tự

Trạng thái và mạch ví dụ. Với DFF, $Q(t + 1) = D(t)$; đáp ứng còn phụ thuộc trạng thái đầu. Chuỗi kiểm thử cần khởi tạo, kích hoạt rồi đưa sai khác ra PO [5]. Mạch ví dụ `seq_example` do nhóm thiết kế có PI A, B, PO Y, sáu cổng và một DFF đồng bộ lý tưởng:

$$N_1 = AQ, \quad N_2 = \neg B, \quad N_3 = N_1 \vee N_2, \quad D = \neg(N_3 A), \quad N_4 = Q \oplus B, \quad Y = N_4 A.$$

Phạm vi gồm 18 lỗi stem (SA0/SA1 trên 9 net), không gộp lỗi, không xét lỗi clock, lỗi bên trong FF hay lỗi nhánh qua DFF.

Trải khung thời gian. Sao khối tổ hợp thành k khung, nối D_t tới Q_{t+1}, quan sát $Y_0, \ldots, Y_{k-1}$. Mỗi lỗi vật lý phải được cấy vào mọi khung. `Q@0` là PI hình thức, không mặc nhiên điều khiển được trên chip. Với trạng thái đầu chưa biết, tập vết PO tốt và lỗi phải rời nhau trên mọi cặp trạng thái đầu; một mẫu chỉ đúng khi $Q@0 = 1$ là có điều kiện. Số cổng tăng theo k; chưa tìm được chuỗi trong k khung không chứng minh untestable với mọi độ dài. Ví dụ N_3/SA0: đặt $(A, B) = (1, 0)$ trong hai khung. Khung 0 có $N_3 = 1/0$, $D = 0/1$ (tốt/lỗi),



Cùng một lỗi vật lý được đặt tại net tương ứng
ở mỗi khung thời gian.

Hình 5.1: Trải ba khung, giữ ràng buộc trạng thái $Q_{t+1} = D_t$.

độc lập Q_0; khung 1 có $Q = 0/1$ và $Y = Q = 0/1$, nên phát hiện bảo đảm. Chuỗi ba khung $(0,0),(1,1),(1,0)$ minh họa riêng pha khởi tạo: khung 0 ép $Q_1 = 1$, khung 1 kích hoạt, khung 2 quan sát.

Full scan. Scan FF chọn dữ liệu chức năng hoặc dữ liệu dịch. Chu trình gồm shift in, capture, shift out; khi cắt FF, Q là pseudo-PI, D là pseudo-PO [6, 7]. Với cùng lỗi N_3/SA0, mẫu $A = 1, B = 0, Q = 0$ cho $D = 0/1$. Một mẫu tổ hợp vẫn cần thời gian dịch và capture. Scan tăng khả năng điều khiển/quan sát nhưng tốn MUX, dây nối và thời gian dịch; partial scan giảm phần cứng nhưng để lại trạng thái không trực tiếp truy cập được. Chế độ chỉ đặt được $Q@0$ vẫn quan sát qua Y, nên khác full scan có đọc D.

Cài đặt và thực nghiệm. Chương trình cung cấp các hàm `full_scan`, `unroll` và `fault_in_frames`; chúng giữ nguyên mạch đầu vào, tạo lại fanout và thứ tự topo. Với full scan, PODEM phát hiện $18/18$ lỗi stem; mọi cube được bộ mô phỏng lỗi và một mô hình vét cạn độc lập xác nhận, bộ mẫu sau điền $X = 0$ phủ $18/18$. Trải khung dùng PODEM kết hợp vét cạn chuỗi bổ sung, đạt $1/18$, $13/18$, $18/18$ khi $k = 1, 2, 3$ từ trạng thái chưa biết; phân nguồn từng hàng ở Chương 7. Kết quả vét cạn độc lập chỉ là mốc đối chiếu, không được gọi là kết quả PODEM.

6. Cài đặt PODEM

Kiến trúc. Lõi chương trình hiện thực PODEM [3], nhận `Circuit`, `Fault`, giới hạn quay lui `max_backtracks` (mặc định 1000) và tùy chọn trace. Sau mỗi lần gán PI, `imply` khởi tạo PI chưa gán bằng X, cấy lỗi và mô phỏng năm giá trị $0, 1, X, D, D'$ theo thứ tự topo; pattern PI chỉ chứa $0, 1, X$. `unroll` (Chương 5) biến mạch tuần tự thành mạch tổ hợp nhiều khung; `run` đọc netlist, gọi PODEM và kiểm tra kết quả bằng bộ mô phỏng lỗi `fault_sim`. Cổng và đầu vào được chọn theo thứ tự netlist ổn định, chưa dùng SCOAP.

Objective và backtrace. Trước kích hoạt, objective yêu cầu net lỗi bằng giá trị ngược stuck-at. Sau kích hoạt, PODEM chọn cổng D-frontier đầu tiên còn X-path tới PO và input X đầu tiên của nó; AND/NAND cần 1, OR/NOR cần 0 để truyền sai biệt. Backtrace đưa mục tiêu về PI: NAND/NOR/NOT đảo giá trị, AND/OR/BUFF giữ nguyên. Với XOR/XNOR, objective lan truyền chọn input X đầu tiên và thử 0; khi backtrace gặp cổng còn đúng một X, giá trị được tính từ parity rail tốt đã biết ($D \mapsto 1$, $D' \mapsto 0$; $q = 0$ với XOR, $q = 1$ với XNOR). Còn nhiều X thì chọn X đầu tiên, thử 0 và để imply tính lại, không giả định các X còn lại bằng 0.

Quay lui, trace và điều kiện dừng. Khi nhánh thất bại, decision stack bỏ các quyết định con, đảo PI gần nhất còn giá trị chưa thử rồi imply lại toàn mạch. Trace xuất một hàng bảy cột cho mỗi lần gán hoặc đảo PI; hàng dẫn tới ngõ cụt ghi `backtrack`, hàng đảo quyết định dùng "—" cho objective/backtrace, khớp quy ước golden trace ở Chương 4. Trace thật trên c17 trùng hai bước ở Bảng 4.1; chương trình có công cụ xuất trace đầy đủ. PODEM trả `DETECTED` khi PO có D/D'; sau kích hoạt, nhánh không còn D-frontier có X-path bị loại. Hết cây quyết định là `UNTESTABLE`; chạm `max_backtracks` là `ABORTED`, không phải `UNTESTABLE`, vì chưa thử hết mọi vector.

Cấy lỗi và fault-frame. Stem tác động mọi fanout; branch chỉ tác động ngõ vào cổng đích. Cấy lỗi giữ rail tốt, ép rail lỗi: $D{+}SA0 = D$, $D{+}SA1 = 1$, $D'{+}SA0 = 0$, $D'{+}SA1 = D'$. Trên mạch trải, imply cấy đồng thời danh sách fault-instance; objective/backtrace lần lượt chọn fault-frame làm mục tiêu. PODEM có thể gán $Q@0$, nên bộ kiểm chứng tuần tự phải phân biệt kết quả có điều kiện và chuỗi bảo đảm, không tự coi trạng thái đầu là điều khiển được.

Kiểm thử và giới hạn. Logic năm giá trị và xử lý lỗi theo mô hình trong giáo trình [1, Ch. 3–4]. Input sai (net/nhánh không thuộc mạch, $SV \notin \{0, 1\}$, giới hạn âm) bị từ chối bằng `ValueError`. Bộ kiểm thử đạt 350 test (Python 3.12.15, pytest 9.1.1), riêng logic và PODEM 255 test. Với c17 $11/SA0$: $X10XX$, 0 backtrack, hai phép gán PI; mạch phụ $t/SA0$: 01, một backtrack (giới hạn 0 trả `ABORTED`). Bộ mô phỏng lỗi độc lập xác nhận các mẫu. Chưa hỗ trợ lỗi nhánh qua DFF, FAN hay heuristic SCOAP.

7. Kết quả và kiểm chứng

7.1 Môi trường, mạch thử và danh sách lỗi

Công cụ viết bằng Python (≥ 3.10), chỉ dùng thư viện chuẩn, kiểm thử bằng `pytest`. Luồng xử lý: trình đọc netlist định dạng `.bench` tính fanout, mức và thứ tự topo; danh sách lỗi; PODEM (Chương 6) sinh pattern; bộ mô phỏng lỗi kiểm chứng; `unroll` (Chương 5) trải mạch tuần tự. Lỗi đầu vào như giá trị kẹt khác 0/1, net hoặc nhánh không tồn tại, số khung không dương đều bị từ chối, không được quy thành kết quả.

Bảng 7.1: Mạch dùng trong thực nghiệm.

Mạch	PI	PO	Cổng tổ hợp	DFF	Nguồn
c17	5	2	6 (NAND)	0	ISCAS'85 [8]
`seq_example`	2	1	6	1	Nhóm tự thiết kế

Với c17, 11 net cho 22 lỗi stem; ba net rẽ nhánh (3, 11, 16), mỗi net hai nhánh, thêm 12 lỗi nhánh: tổng **34 lỗi gốc**. Gộp tương đương theo từng loại cổng [1, pp. 12–14] (với NAND, SA0 ở ngõ vào tương đương SA1 ở ngõ ra) tạo sáu lớp ba lỗi, còn **22 lỗi đại diện**. Tập 22 đại diện này *khác* tập 22 lỗi stem, dù trùng số; lỗi mẫu `11/SA0` không thuộc lớp nào nên được giữ. Mạch tuần tự dùng 18 lỗi stem vật lý (9 net), cấy vào mọi khung, không gộp và không có lỗi nhánh.

7.2 Phương pháp kiểm chứng

Mọi pattern của PODEM đều được mô phỏng lại bằng bộ mô phỏng lỗi nối tiếp độc lập [1, Ch. 3]: tính mọi net theo thứ tự topo cho mạch tốt và mạch lỗi; lỗi stem ép cả net, lỗi

nhánh chỉ ép ngõ vào của cổng nhận nhánh; phát hiện nếu một PO khác nhau. Bản mô phỏng song song theo pattern (phép toán bit) cho cùng kết quả trên toàn bộ 32 vector c17. Theo quy ước nhóm, X được điền 0; ngoài ra một pattern có X chỉ được xác nhận khi đã vét cạn *mọi* cách điền (kết quả: có, không, hoặc chưa kiểm chứng hết khi quá 16 bit X). Kết luận `UNTESTABLE` được đối chiếu bằng vét cạn khi mạch có tối đa 16 PI. Bộ sinh chỉ chuyển sang vét cạn khi PODEM báo chưa hỗ trợ, và ghi nguồn theo từng hàng. Nén tập test giữ pattern phát hiện thêm lỗi mới rồi duyệt ngược để bỏ pattern thừa [1, Ch. 4]. Các chỉ số: fault coverage $= DT/|F|$, test coverage $= DT/(|F| - UT)$, fault efficiency $= (DT + UT)/|F|$, với DT là số lỗi phát hiện, UT số lỗi chứng minh không thể phát hiện.

7.3 Kết quả trên c17

Bảng 7.2: Kết quả PODEM trên c17; mọi pattern được fault simulation xác nhận.

Chỉ số	22 lỗi đại diện	34 lỗi gốc
Fault coverage	22/22 = 100%	34/34 = 100%
Test coverage, fault efficiency	100%, 100%	100%, 100%
`ABORTED`; backtrack trung bình	0; 0	0; 0
Pattern trước / sau nén	22 / 6	34 / 7
Tập sau nén phủ 34 lỗi gốc	34/34	34/34
Pattern bị kiểm chứng sai	0	0

PODEM tìm được pattern cho mọi lỗi không cần backtrack, mọi pattern đúng với mọi cách điền X, và tập 6 pattern sau nén phủ cả 34 lỗi gốc. Lệnh, môi trường và thời gian từng lần chạy được ghi kèm kết quả để tái lập. Với `11/SA0`, PODEM cho $X10XX$ (thứ tự PI $1, 2, 3, 6, 7$) sau hai bước, đúng golden trace Chương 4; 01000 cho đầu ra tốt $(1, 1)$ và lỗi $(0, 0)$. Bộ mô phỏng cũng xác nhận các cube chạy tay của D-algorithm ($4/4$ và $8/8$ cách điền phát hiện lỗi tại 22) và mạch phụ ở Chương 4 (chỉ vector 01 phát hiện `t/SA0`; PODEM cần một backtrack, giới hạn 0 cho `ABORTED`).

7.4 Mạch tuần tự

Với full scan, PODEM phát hiện $18/18$ lỗi stem, trùng mô hình vét cạn độc lập (Chương 5). Khi không scan, mạch được trải k khung và $Q@0$ là trạng thái đầu *chưa biết*: mạch tốt và mạch lỗi có thể bắt đầu ở trạng thái bất kỳ. Chuỗi chỉ được tính là phát hiện *bảo đảm* khi mọi vết PO của mạch tốt khác mọi vết PO của mạch lỗi, với mọi cặp trạng thái đầu và mọi cách điền X. PODEM coi $Q@0$ là PI hình thức nên có thể gán nó; chuỗi chỉ đúng khi $Q@0$ có giá trị cụ thể là kết quả *có điều kiện*. Khi pattern PODEM phụ thuộc $Q@0$, công cụ tìm chuỗi bảo đảm bằng vét cạn trên các PI điều khiển được và ghi nguồn theo từng hàng.

Bảng 7.3: `seq_example`, 18 lỗi stem, trạng thái đầu chưa biết.

k	Bảo đảm	Nguồn chuỗi bảo đảm (PODEM / vét cạn)	Chỉ có điều kiện $Q@0$	Nếu đặt được $Q@0$
1	1/18	0 / 1	9	10/18
2	13/18	3 / 10	5	18/18
3	18/18	3 / 15	0	18/18

Số phát hiện bảo đảm (1, 13, 18) trùng kết quả vét cạn độc lập. Với hai khung, 5 lỗi chỉ phát hiện khi biết $Q@0$, nên không gọi hai khung là 100% từ trạng thái chưa biết. Với $k = 1$, 8 lỗi được PODEM kết luận không thể phát hiện trong một khung và vét cạn xác nhận; đây là giới hạn của k, chưa phải `untestable`. Cột cuối ứng với giả định điều khiển được trạng thái đầu: chỉ đặt được trạng thái đầu, còn quan sát vẫn qua PO ở các khung, nên *không* tương đương full scan (full scan đặt Q và đọc D ở mọi pattern).

7.5 Nhận xét và giới hạn

Bộ mô phỏng độc lập giúp phát hiện sai lệch khi tích hợp: trên phiên bản PODEM trước khi hoàn thiện, nó chỉ ra backtrace chưa hỗ trợ cổng XOR và bốn kết luận `UNTESTABLE` sai trên mạch đã trải; bản cuối không còn sai lệch nào. Giới hạn: c17 nhỏ nên coverage 100% chưa nói lên hiệu năng; gộp lỗi mới ở mức tương đương, chưa gộp trội; vét cạn tham chiếu chỉ dùng được với mạch ít PI; chế độ tuần tự chưa có lỗi nhánh, kết quả phụ thuộc k, và phần lớn chuỗi bảo đảm vẫn lấy từ vét cạn vì PODEM gán $Q@0$. Hướng cải tiến là cho PODEM giữ $Q@0 = X$ để tự sinh chuỗi bảo đảm.

8. Kết luận

Nhóm đã tích hợp nguyên lý, D-algorithm, PODEM, full scan/trải khung, lõi Python và mô phỏng lỗi độc lập. Trên c17, PODEM đạt $34/34$ lỗi gốc hoặc $22/22$ đại diện; nén từ 34 xuống 7 hoặc 22 xuống 6 pattern, cả hai tập nén phủ $34/34$ lỗi gốc. Full scan đạt $18/18$ lỗi stem; CLI tuần tự kết hợp PODEM và vét cạn chuỗi bổ sung đạt $1/18$, $13/18$, $18/18$ tại $k = 1, 2, 3$ từ trạng thái đầu chưa biết. Trace tay và code thống nhất; bốn lần chọn cube D-algorithm và hai phép gán PI PODEM khác đơn vị, không suy ra tăng tốc. Kết quả chỉ áp dụng stuck-at trên các mạch nhỏ, không bảo đảm mọi khuyết tật vật lý hay hiệu năng trên mạch lớn. Hướng tiếp theo là sinh chuỗi không phụ thuộc $Q@0$, heuristic FAN, scan compression và lỗi trễ.

Tài liệu tham khảo

[1] Laung-Terng Wang, Cheng-Wen Wu, và Xiaoqing Wen, biên tập. *VLSI Test Principles and Architectures: Design for Testability*. Morgan Kaufmann, 2006. ISBN: 978-0-12-370597-6.

[2] J. Paul Roth. "Diagnosis of Automata Failures: A Calculus and a Method". In: *IBM Journal of Research and Development* 10.4 (1966), pp. 278–291. DOI: 10.1147/rd.104.0278.

[3] Prabhakar Goel. "An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits". In: *IEEE Transactions on Computers* C-30.3 (1981), pp. 215–222. DOI: 10.1109/TC.1981.1675757.

[4] Hideo Fujiwara và Takeshi Shimono. "On the Acceleration of Test Generation Algorithms". In: *IEEE Transactions on Computers* C-32.12 (1983), pp. 1137–1144. DOI: 10.1109/TC.1983.1676174.

[5] NPTEL. *ATPG for Synchronous Sequential Circuits*. URL: https://archive.nptel.ac.in/content/storage2/courses/106103016/module10/lec1/1.html (visited on 10/04/2026).

[6] Edward B. Eichelberger và Thomas W. Williams. "A Logic Design Structure for LSI Testability". In: *Proceedings of the 14th Design Automation Conference*. 1977, pp. 462–468. URL: https://ieee-ceda.org/media/logic-design-structure-lsi-testability (visited on 10/04/2026).

[7] NPTEL. *Scan Chain Based Sequential Circuit Testing*. URL: https://archive.nptel.ac.in/content/storage2/courses/106103016/module10/lec2/5.html (visited on 10/04/2026).

[8] Franc Brglez và Hideo Fujiwara. "A Neutral Netlist of 10 Combinational Benchmark Circuits and a Target Translator in FORTRAN". In: *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*. 1985.